

Data Analysis with Python 2024–2025

Parte 3: NumPy (Parte 1)

Soluções dos Exercícios

Tradução e Adaptação: Universidade de Helsinque (MOOC.fi)

Nota Importante

Este material é uma tradução e adaptação baseada no curso *Data Analysis with Python 2024-2025*, oferecido pela **Universidade de Helsinque (MOOC.fi)**. As soluções apresentadas a seguir foram elaboradas para fins didáticos, seguindo os enunciados dos exercícios propostos no curso original.

Conteúdo

1 Soluções - Capítulo 2 (NumPy)

Exercício 2.11 – Linhas e colunas

Solução proposta

```
import numpy as np

def get_rows(a):
    return [a[i, :] for i in range(a.shape[0])]

def get_columns(a):
    return [a[:, j] for j in range(a.shape[1])]

def main():
    a = np.array([[5, 0, 3, 3],
                  [7, 9, 3, 5],
                  [2, 4, 7, 6],
                  [8, 8, 1, 6]])

    print(get_rows(a))
    print(get_columns(a))

if __name__ == "__main__":
    main()
```

Observação: uma alternativa equivalente, usando a transposição, é implementar `get_columns` como `get_rows(a.T)`, já que transpor a matriz troca linhas por colunas:

```
def get_columns_alt(a):
    return get_rows(a.T)
```

Exercício 2.12 – Vetores-linha e vetores-coluna

Solução proposta

```
import numpy as np

def get_row_vectors(a):
    return [a[i:i+1, :] for i in range(a.shape[0])]

def get_column_vectors(a):
    return [a[:, j:j+1] for j in range(a.shape[1])]

def main():
    a = np.array([[5, 0, 3],
                  [3, 7, 9]])
    print("Vetores-linha:")
    print(get_row_vectors(a))
    print("Vetores-coluna:")
    print(get_column_vectors(a))

if __name__ == "__main__":
    main()
```

Observação: o truque usado aqui é indexar com uma fatia de tamanho 1 (`i:i+1`) em vez de um único índice inteiro (`i`) – assim o NumPy preserva aquela dimensão em vez de eliminá-la, o que é exatamente o que diferencia este exercício do anterior.

Exercício 2.13 – Diamante

Solução proposta

```
import numpy as np

def diamond(n):
    # metade superior: identidade seguida da identidade espelhada
    # horizontalmente (concatenadas lado a lado)
    identidade = np.eye(n, dtype=int)
    metade_superior = np.concatenate(
        (identidade, np.fliplr(identidade)),
        axis=1
    )
    # metade inferior: espelho vertical da metade superior,
    # sem repetir a linha central (que já está na metade superior)
    metade_inferior = np.flipud(metade_superior)[1:]
    return np.concatenate((metade_superior, metade_inferior))

def main():
    print(diamond(3))
    print(diamond(1))

if __name__ == "__main__":
    main()
```

Exercício 2.14 – Comprimento de vetores

Solução proposta

```
import numpy as np

def vector_lengths(a):
    return np.sqrt(np.sum(a**2, axis=1))

def main():
    a = np.array([[3, 4],
                  [1, 0],
                  [0, 0]])
    print(vector_lengths(a))
    # [5.  1.  0.]

if __name__ == "__main__":
    main()
```

Exercício 2.15 – Ângulos entre vetores

Solução proposta

```
import numpy as np

def vector_angles(X, Y):
    produto_interno = np.sum(X * Y, axis=1)
    norma_x = np.sqrt(np.sum(X**2, axis=1))
    norma_y = np.sqrt(np.sum(Y**2, axis=1))
    cos_alpha = produto_interno / (norma_x * norma_y)
    cos_alpha = np.clip(cos_alpha, -1.0, 1.0)
    return np.degrees(np.arccos(cos_alpha))

def main():
    X = np.array([[1, 0], [1, 1], [1, 0]])
    Y = np.array([[0, 1], [1, 0], [1, 0]])
    print(vector_angles(X, Y))
    # [90. 45.  0.]

if __name__ == "__main__":
    main()
```

Exercício 2.16 – Tabuada revisitada

Solução proposta

```
import numpy as np

def multiplication_table(n):
    linhas = np.arange(n).reshape(n, 1)  # vetor-coluna: 0, 1, ..., n-1
    colunas = np.arange(n).reshape(1, n)  # vetor-linha: 0, 1, ..., n-1
    return linhas * colunas               # broadcasting: (n, 1) * (1, n) -> (n, n)

def main():
    print(multiplication_table(4))

if __name__ == "__main__":
    main()
```